

Network Intrusion Detection Based on Graph Neural Network and Ensemble Learning

Lina Zhang¹, Yang Wang¹, Minyue Wu¹, Yanwei Wang^{2,*}, Ying Zheng¹

¹ The Belt and Road Joint Laboratory on Measurement and Control Technology, School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, 430074, China

² School of Mechanical & Electrical Engineering, Wuhan Institute of Technology, 693 Chuxiong Road, Wuhan, 430073, China
Email: ywwang.cad@gmail.com

Abstract—Network technology has become an inseparable part of daily life, which is easily attacked by some criminals. Therefore, ensuring the security of network becomes an urgent problem. In the past few years, various deep learning techniques have been put forward for identifying network intrusions. However, the majority of these approaches overlook the non-Euclidean nature of relationships within network intrusion data. In this paper, graph neural network (GNN) is used to learn the relationship between data. The sample balancing module is to address the significant disparity between the number of intrusive samples and normal samples. The results obtained from the experiments indicate that the suggested approach can enhance the effectiveness of the intrusion detection system.

Keywords—intrusion detection, graph neural network, node classification, CICIDS 2017

I. INTRODUCTION

In the midst of the significant progress in networking during the 21st century, the internet has progressively gained a pivotal role in various aspects of human daily life. Activities like online education, QR code payments, video gaming, and autonomous driving exemplify this trend. As a consequence, maintaining the normal operation of the network has become a central area of research focus. The inception of the intrusion detection (ID) concept dates back to 1980, when James Anderson introduced the idea for the first time [1]. Initially, intrusion detection focused on examining the attributes of network assaults and devising corresponding rules for identification. However, these early intrusion detection methodologies struggle to precisely extract relevant insights from the progressively varied dataset. As a result, it leads to a reduced rate of correctly identifying intrusions and an elevated occurrence of false positives.

In modern times, machine learning techniques such as K-Means [2], support vector machine (SVM) [3], random forest (RF) [4], and cyclic neural networks (LSTM) [5], are introduced to intrusion detection to address network data with high dimension and large numbers. Although the detection accuracy of these methods has been improved compared with traditional methods, they do not consider the on-euclidean relationship between data. The images and samples processed by traditional neural networks (NNs) are data in euclidean space with regular data structure. GNN process non-Euclidean graph data with extremely irregular structure. Graph data has

a stronger data and knowledge representation than images and samples. The nodes in the graph are connected by edges to effectively and adequately represent information such as the relationship between different samples, which can maximize the structural features of realistic graphs [6] and thus improve the detection accuracy. GNN have found extensive application across diverse domains, including but not limited to taxi demand forecasting [7], social networks [8], ID [9], etc.

Furthermore, a practical challenge arises from imbalanced sample distribution, leading to a predisposition of current methodologies toward the class that is more heavily represented. In order to tackle the aforementioned concerns, this paper introduces a comprehensive learning model featuring both a sample balancing module and an adjacency matrix reconstruction module. GraphSAGE, a standard model of graph neural networks, is used as the Blending base classifier and the overall model is validated on a typical sample imbalanced dataset - CICIDS 2017. In comparison to using a single technique, the integrated learning approach enhances both the intrusion detection rate and reduces the false positive rate. The sample balancing module takes the raw intrusion data, performs a minority class sample generation and a majority class sample sampling operation. The adjacency matrix reconstruction module reorganises the adjacency matrix using Euclidean distances to form the final graph structure dataset.

The subsequent sections of this paper are organized as follows: Section 2 offers a concise overview of relevant prior research. Section 3 outlines the research methodology adopted in this study. In Section 4, the comparative experimental outcomes of the proposed techniques are presented. The concluding remarks along with directions for future work are provided in Section 5.

II. RELATED WORK

The packet data captured in the network contains both normal pattern and abnormal pattern. ID aims to detect the abnormal pattern from the packet data. Generally, sample unbalance exists in the packet, that is normal data is numerous while abnormal data is very few. For example, in a public dataset CICIDS 2017, there are millions of normal samples and only a few tens or thousands of abnormal samples. In order to address the issue of imbalanced samples, numerous

approaches have been put forth, which can be categorized into two levels: the sample level and the algorithm level.

The problem is solved at the sample level, using changing the amount of data in the original category. Tan et al. [10] presented an oversampling based on SMOTE, generate a few intrusive-type sample data, and utilized RF to detect network intrusion. Cui et al. proposed an IDS [11] with an imbalance processing module called GMM-WGAN. The imbalance processing module combines GMM and WGAN to generate minority attack samples, thus expanding the set of marked samples. The majority class was downsampled using the GMM-based clustering algorithm.

This problem can also be solved at the algorithmic level by improving the majority and minority class classification accuracy while keeping the number of different classes constant. Bagging or boosting ensemble learning algorithms have been used to improve the learning of the algorithm on unbalanced data and hence the final classification accuracy. In the realm of intrusion detection, Jung et al. [12] utilized a combination of hybrid resampling approaches and bagging to address the challenge of imbalanced data distribution. Cost-sensitive methods improve the classification performance of imbalanced data by assigning different misclassification costs to the majority and minority classes of a dataset. Gupta et al. [13] devised a three-layer detection algorithm. In the first layer, a cost-sensitive NN is used to separate normal traffic from suspicious network traffic, identifying majority and minority attacks with high attack detection rates.

Nonetheless, the foundational data structure employed in these studies remains confined to grid data or sequential data. Traditional NNs only learn the structured data in euclidean space. GNN can learn the topological relationships in networks and obtain better information feature representation. Duan et al. [14] designed a dynamic GNN for intrusion detection, which converted network packets into dynamic space-time graphs, and trained the network based on semi-supervised learning ideas. Therefore, it is significant to consider non-euclidean relationships while addressing the sample imbalance.

III. MATERIALS AND METHODS

This section outlines the distinctive framework of network intrusion detection which relies on the integration of GNN and ensemble learning. This architecture is visually represented in Figure 1. The presented structure encompasses four core components: data preprocessing, sample balancing, adjacency matrix reconstruction, and ensemble learning modules.

A. CICIDS 2017 dataset

Presently, the KDDCUP 99 dataset and its evolutionary version, the NSL-KDD dataset, stand as the most extensively utilized datasets in intrusion detection systems. Nonetheless, these datasets are outdated and exhibit certain omissions in terms of prevalent attack types, which results in an incomplete representation of contemporary intrusion trends. Therefore, the CICIDS 2017 dataset was finally selected as the validation

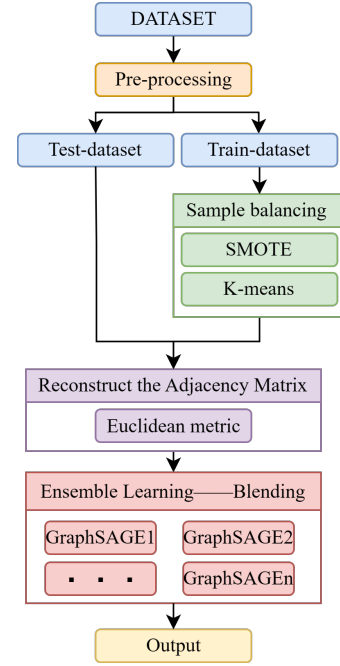


Fig. 1. Overall flow of algorithm

dataset for our algorithm. The CICIDS 2017 dataset [15] encompasses a variety of network traffic data, including regular patterns as well as recent attack instances such as botnet attacks, Denial of Service (DoS), Distributed Denial of Service (DDoS), PortScan, FTP-Patator, and SSH-Patator.

The dataset mainly contains data information from Monday to Friday, with Monday containing normal traffic data and Tuesday to Friday containing abnormal intrusion data and normal traffic. As the intrusions and attacks are different every day and contain missing values. So the data from each day was combined and the missing values were removed in preparation for the subsequent construction of the graph dataset. The overall data was divided into a test dataset, a validation dataset and a training dataset. The proportion of the test dataset was 30% of the overall dataset respectively. The statistical results of all data are shown in table I.

B. Data Preprocessing Module

a) *Label-encoding*: The six columns ‘FlowID’, ‘SourcePort’, ‘DestinationPort’, ‘Timestamp’, ‘SourceIP’ and ‘DestinationIP’ in the original data features, which mainly serve the purpose of the original network identification. These six columns are not useful for the subsequent network classification training, so they are removed. The original dimensionality was reduced to 79 dimensions. The label columns in the data consisted of character class data, so class coding was used to convert the non-numeric data into numbers for subsequent training and testing. The coding correspondence between the classes is shown in the first and second columns of table I.

b) *Feature Normalization*: Following the pre-processing step, the subsequent action involves feature normalization through the utilization of the standard deviation normalization

TABLE I
ALL ATTACK TYPES AND NORMAL TRAFFIC WITHIN THE CICIDS 2017
DATASET

No.	New labels	Old labels	Verification	Test	Training
0	Normal	Benign	476,977	681,396	1,112,947
1	Botnet	Bot	411	587	958
2	DoS/DDoS	DDoS DoS Golden Eyes DoS Hulk DoS-Slowloris DoS-Slowhttptest Heartbleed	79,747	113,924	186,077
3	Infiltration	Infiltration	7	11	18
4	Port Scan	Port Scan	33,349	47,641	77,814
5	Brute Force	FTP-Patator SSH-Patator	2,905	4,150	6,777
6	Web Attack	Web Attack-Brute Force Web Attack-XSS Web Attack-SqlInjection	458	654	1,068

formula provided in Eq. (1). The attributes within the network intrusion dataset exhibit considerable variability, and by employing standard deviation normalization, the processed data is transformed to adhere to a standard normal distribution, where the mean is 0 and the standard deviation is 1. This is done to maintain a consistent ratio of all attributes and to prevent the number of features from being too large, an important step in the classification algorithm.

$$X^* = \frac{X - \mu}{\sigma} \quad (1)$$

Here, μ represents the average of all data within the network intrusion dataset, while σ stands for the standard deviation.

C. Sample Balancing Module

The distribution of samples during network intrusion detection exhibits a noticeable imbalance, which is a reflection of real-world conditions. Extracting rules solely from a limited number of intrusion events presents a challenge. Even if a classification model is developed, there's a risk of excessive dependence on a small subset of data, potentially causing overfitting. Consequently, a strategy for balancing samples is implemented on the training dataset. The primary approach involves reducing the sample count of the majority class while augmenting the sample count of the minority class.

Firstly, the SMOTE is used to extend the 1, 3 and 6 class intrusions. The fundamental concept behind SMOTE involves analyzing and characterizing multiple categories, then introducing synthetic samples into the dataset through simulation. This process aims to mitigate any noticeable skewness that might exist among the original categories' frequencies. The implementation process of the algorithm makes use of KNN technology. The steps for generating new samples are as follows: First, a positive sample is selected and the KNN algorithm is used to calculate the K nearest neighbours for each minority sample. During the second step, N samples are chosen at random from the K nearest neighbors. These samples are then subject to random linear interpolation, as depicted in

Equation (2). The third step is to construct a new minority 1-class sample. In the fourth stage, the newly generated samples are integrated with the existing data to create a fresh training dataset.

$$x_{new} = x + rand(0, 1) * (\tilde{x} - x) \quad (2)$$

Where x is data from classes 1, 3 and 6; $\tilde{x}$ is a random sample of points selected from the k nearest neighbours of x ; and x_{new} is the new generated sample.

As a result of these operations, a small number of classes of intrusion sample data were generated. The next step is to reduce the sample data size by selecting some representative data from the normal data and the majority class data using the k-means clustering algorithm. K-means is a type of unsupervised learning that automatically groups similar objects into the same clusters without prior knowledge of the classes.

The steps of the k-means algorithm are as follows: In the first step, K objects are selected from the original target data as the initial clustering centres $a = \{a_1, a_2, \dots, a_k\}$; In the second step, the distance between each sample and the k clustering centres is calculated and it is assigned to the cluster corresponding to the cluster centre with the smallest distance; The third step, for each a_j , recalculate the clustering center $a_j = \frac{1}{|C_j|} \sum_{x \in C_j} x$. Iterate through steps 2 and 3 as specified until the designated number of iterations is achieved.

D. The Adjacency Matrix Reconstruction Module

A graph can be represented as $G = (V, E)$, which consists of v nodes, $V = \{v_1, v_2, \dots, v_n\}$ and a set of edges E . Denote the edge from node $u \in V$ to node $v \in V$ as $(u, v) \in E$. In this article, an undirected property graph is built. The points of the undirected graph represent each piece of data in the network intrusion dataset, and the edges represent the connections among the samples within the dataset. The adjacency matrix ($A = (a_{ij}) \subseteq R^{N \times N}$) is the basis for forming the graph structure. The essence of the matrix is the presence of edges and the ranks of the matrix represent the ordering of the nodes. In an adjacency matrix, a value of 1 indicates that there is an edge between two nodes, while a value of 0 indicates that two nodes are not connected.

According to the principle of homogeneity [16], [17], it is believed that sample points of the same category should be spatially related. Euclidean distance serves as a metric to gauge the direct separation between two points within a two-dimensional plane, representing the most intuitive and concise straight-line distance between them. Hence, the utilization of Euclidean distance lies in assessing similarity, resulting in the derivation of both the distance matrix *Distance* and the similarity matrix *S*.

$$Distance = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (3)$$

$$S = \frac{1}{1 + Distance} \quad (4)$$

Considering the realm of complex networks, a significant number of real-world networks tend to demonstrate sparsity. However, the elements within the previously acquired similarity matrix all fall within the range of $[0, 1]$, leading to a densely populated similarity matrix S . Therefore, the symmetric sparse adjacency matrix A needs to be extracted from the similarity matrix S . In this context, we introduce the idea of controlling network sparsity through the utilization of a threshold $\varepsilon > 0$. In other words, when the value within the similarity matrix S is smaller than ε , it is modified to 0 in the adjacency matrix A . It can be expressed by the Eq. (5).

$$A_{ij} = \begin{cases} 1, S_{ij} > \varepsilon \\ 0, S_{ij} < \varepsilon \end{cases} \quad (5)$$

E. Ensemble Learning Module

At its core, ensemble learning involves amalgamating multiple individual learners to create a novel learner. The fundamental concept underlying ensemble learning is that in cases where one base learner makes an erroneous prediction, the other base learners have the potential to rectify this mistake. Since the sample balancing module censors majority classes of samples from the training dataset, the utilisation of the data is reduced to some extent. Therefore, training multiple base learners to learn the network intrusion data can improve the data utilisation and the final detection accuracy. In this paper, the base learner uses GraphSAGE in GNN, the integration learning idea uses blending, and the ensemble learner uses decision trees (DT).

a) *Message Passing*: GNN stand as one of the most recent and rapidly growing subfields in machine learning. Its strength and potential derive from its capacity to leverage the intrinsic graphical architecture of the extensive datasets encountered within real-world application domains. Structural information is obtained by modelling a set of objects and their relationships [18]. GNNs differ from ordinary neural networks in that the original data features are learned while the data is updated by establishing connections between nodes. Ultimately a low-dimensional synthesis of the embedded information is obtained for the final classification task.

This step of the spatial GNN relies on a messaging scheme, which consists of a message, aggregation and update step. The messages sent to the nodes by each neighbour are determined by the message function (ϕ). The aggregation function ($\square$) aggregates these incoming messages into a single message, and the aggregated message can be obtained by averaging or maximising over neighbours. Finally, the update function (γ) updates the representation of the aggregated neighbour message to the target node. In summary, the K -th layer of GNN computes the hidden representation of the node by:

$$h_{N_i}^{(k)} = \square_{j \in N_i} \phi^{(k)}(h_i^{(k-1)}, h_j^{(k-1)}, e_{j,i}) \quad (6)$$

$$h_i^{(k)} = \gamma^{(k)}(h_i^{(k-1)}, h_{N_i}^{(k)}) \quad (7)$$

As shown in Figure 2, the picture shows a complete $k = 2$ message passing process. Node information is collected progressively, starting from the outermost neighbor of the target node and moving inwards towards the target node. When the target node is A in the figure, its neighbor node is first found to form set N_i . The neighbor node of target node A is $\{B, C, D\}$, and the neighbor node of corresponding node B is $\{A, C\}$. When the node of B is aggregated and updated, it will be aggregated with the other updated nodes of node A into new node information.

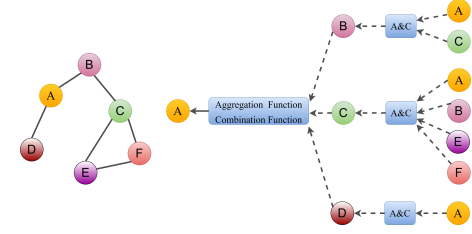


Fig. 2. Message aggregation diagram

b) *GraphSAGE*: Most graph embedding frameworks are transductive learning, which is equivalent to generating node embedding for a fixed graph, where entire data collection is available in training. The process is to learn from this fixed graph. Once some nodes in the graph have changed, the whole graph needs to be retrained and re-learned. Therefore, the transductive learning cannot generate node embeddings for new nodes that are not in the graph. In general, network intrusion detection always detects future unknown intrusion types by learning existing intrusion behaviour data, so it needs a method to infer future unknown information.

Therefore, the GraphSAGE model, which is a model of inductive learning, is used. Inductive learning means that data that cannot be seen during training can be computed directly without relearning the entire graph [19], rather than training a separate embedding for each node. Instead, a learned function is utilized to create an embedding by sampling and combining the features of the neighboring nodes in a node's vicinity. The GraphSAGE model undoubtedly holds broader applicability due to the following reasons:

First, changing nodes does not require relearning the entire graph, which significantly reduces computation; Second, it can infer previously unseen data, which will greatly increase its application value; Third, the utilization of the neighbor sampling technique addresses the issue of memory overload within GCN and proves compatible with extensive-scale graph structures; Fourthly, by transforming the singular direct push node that solely depicts a single local structure into a node-inductive representation that aligns with various local structures, it can proficiently curb training overfitting while bolstering the capability for generalization. The proposed method uses GraphSAGE to classify the nodes of the oversized graph generated from the network invasion dataset. The nodes of the K layer are calculated as in Eq. (8), (9), (10), with the input raw node characterisation using the 79-dimensional features

of the nodes from the training set.

$$h_{N(v)}^k = AGG_K(\{h_u^{k-1}, \forall u \in N(v)\}) \quad (8)$$

$$h_v^k = \sigma(W^k \bullet CONCAT(h_v^{k-1}, h_{N(v)}^k)) \quad (9)$$

$$Z_v = h_v^k, \forall v \in V \quad (10)$$

Here, $N(v)$ represents the set of neighbors for node v , and h_u^{k-1} corresponds to the representation of the neighboring node in the preceding layer.

c) *Blending-DT*: The process of implementing blending is as follows: First, the network intrusion dataset was divided into a training set and a test set, and 30% of the training set was extracted from the training set as the validation set. After the three modules mentioned above, a new training set was obtained after the training set was sample balanced. The three datasets constitute the node graph data. Secondly, multiple base learners, namely GraphSAGE with different parameters, are created. Training is performed using the training set and testing is performed using the validation and test sets to obtain validation and test results. Third, DT was used as the second layer learner, and the verification results of multiple base GraphSAGE were taken as the DT training data, and the test results of base learner were taken as the DT test input. The DT result is taken as the final classification result.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

a) *Evaluation Index*: To evaluate the effect of classified tasks, there is no getting rid of the confusion matrix. The obfuscation matrix displays the predicted and real categories of data in matrix form, as shown in Table II. In the matrix, the rows correspond to actual values, while the columns correspond to predicted values. True positive (TP) signifies the count of normal objects that the model correctly identifies as normal. A False positive (FP) occurs when a model mistakenly labels a small number of normal samples as attacks. True negative (TN) signifies the count of attack attempts accurately identified by the model as attacks. False negative (FN) refers to the count of attack instances that the model incorrectly categorizes as normal.

TABLE II
CONFUSION MATRIX FORM

Confusion Matrix		Predicted value	
		Positive	Negative
True value	Positive	TP	FP
	Negative	FN	TN

The primary evaluation metrics encompass the following: F1-score, recall rate, precision, and accuracy. These metrics are utilized to assess and quantify the model's performance. The four basic attributes to measure model performance are defined as follows:

$$Recall = \frac{TP}{TP + FN} \quad (11)$$

$$Precision = \frac{TP}{TP + FP} \quad (12)$$

$$F1 - score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (13)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (14)$$

b) *Experimental Parameters*: After processing by sample balancing module, the number of intrusion and normal samples of each kind reached 6700 ± 100 . The reconstruction adjacency matrix of the training test graph data set uses the same threshold value ε . The training subgraph contains a total of 3000 nodes. The number of nodes in each subgraph of the test set is relatively consistent with the size of the training subgraph. The detailed hyperparameters of the model are displayed in Table III.

TABLE III
MODEL SPECIFIC PARAMETER

Model	hyperparameter
subgraph_nodes	3000
SMOTE	sampling_strategy = 6700
k-means	n_clusters = 670
GraphSAGE_1	h_feats = 16
GraphSAGE_2	h_feats = 32
GraphSAGE_3	h_feats = 64
ε	0.9
DT	class_weight = balanced criterion = gini min_impurity_decrease = 0.07 splitter = best

c) *Experimental results*: This section explores the importance and comparison of the proposed integration model and clarifies the significance of the introduced intrusion detection model by comparing intrusion detection systems based on integrated mechanisms. The comparative analysis of the performance outcomes of the proposed model in relation to existing models, focusing on metrics such as precision, recall, and F1-score. In the study conducted by Sharafaldin et al. [20], three fundamental classifiers -Adaboost, Multi-Layer Perceptron (MLP), and Parsimonious Bayes (NB) - were employed to perform multi-classification tasks using the CICIDS 2017 dataset. Karatas et al. [21] applied a combination of SMOTE and subsequent linear discriminant analysis (LDA) to address the task of balancing classes. lee and Park et al. [22] used SMOTE followed by RF classifier to reduce the imbalance between attack classes. In Table IV, the outcomes demonstrate a noteworthy enhancement in precision, recall, and F1-score of the intrusion detection model through the implementation of the integrated approach. The intrusion detection system's precision stands at 0.96, accompanied by a recall rate of 0.94 and an F1 score of 0.95. This underscores the considerable effectiveness of the proposed integrated model in enhancing intrusion detection performance.

TABLE IV
A COMPARISON OF MULTIPLE CATEGORIES AND EXISTING WORK.

<i>Method</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-score</i>
AdaBoost	0.77	0.84	0.81
MLP	0.77	0.83	0.79
NB	0.88	0.04	0.04
SMOTE+LDA	0.91	0.91	0.91
SMOTE+RF	0.88	0.87	0.87
Proposed	0.96	0.94	0.95

V. CONCLUSIONS

This paper introduces a novel approach to network intrusion detection that combines ensemble learning and GNN. This methodology effectively takes into account non-Euclidean relationships within the data while also tackling the issue of imbalanced samples. The proposed framework has four modules: data preprocessing module, sample balancing module, the adjacency matrix reconstruction module and ensemble learning module. The sample balancing module and the ensemble learning module address the problem of low accuracy of minority class classification due to the small number of intrusion samples in network intrusion. The adjacency matrix reconstruction module creates similar links for the data. Additionally, validation was conducted using the CICIDS 2017 dataset, revealing that the suggested approach has exhibited enhanced precision in comparison to prior methodologies. In the future, other GNN models and the construction of adjacency matrices will be investigated to further refine the model.

REFERENCES

- [1] J. P. Anderson, "Computer security threat monitoring and surveillance," 1980.
- [2] C. Liu, Z. Gu, and J. Wang, "A hybrid intrusion detection system based on scalable K-Means+ random forest and deep learning," *IEEE Access*, vol. 79, pp. 75729-75740, 2021.
- [3] M. Mohammadi, T. A. Rashid, S. H. T. Karim, A. H. M. Aldalwie, Q. T. Tho, Moazam Bidaki, et al, "A comprehensive survey and taxonomy of the SVM-based intrusion detection systems," *Journal of Network and Computer Applications*, vol. 178, pp. 102983, 2021.
- [4] A. Assiri, "Anomaly classification using genetic algorithm-based random forest model for network attack detection," *CMC-COMPUTERS MATERIALS & CONTINUA*, vol. 66, no. 1, pp. 767-778, 2021.
- [5] S. M. Sohi, J. P. Seifert, and F. Ganji, "RNNIDS: Enhancing network intrusion detection systems through deep learning," *Computers & Security*, vol. 102, pp. 102151, 2021.
- [6] A. Grover, and J. Leskovec, "node2vec: Scalable feature learning for networks," *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 855-864, 2016.
- [7] J. Y. Xu, S. Zhang, C. C. Wu, and Q. L. Yuan, "Application of an Adaptive Adjacency Matrix-Based Graph Convolutional Neural Network in Taxi Demand Forecasting," *Mathematics*, vol. 10, no. 19, pp. 3694, September 2022.
- [8] X. Yin, W. Lin, K. Sun, C. Wei, and Y. Chen, "A2S2-GNN: Rigging GNN-Based Social Status by Adversarial Attacks in Signed Social Networks," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 206-220, 2023.
- [9] P. Biswas, T. Samanta, and J. Sanyal, "Intrusion detection using graph neural network and Lyapunov optimization in wireless sensor network," *Multimedia Tools and Applications*, vol. 82, pp. 14123-14134, 2023.
- [10] X. Tan, S. Su, Z. Huang, X. Guo, Z. Zuo, X. Sun, et al, "Wireless sensor networks intrusion detection based on SMOTE and the random forest algorithm," *Sensors*, vol. 19, no. 1, pp. 203, 2019.
- [11] J. Cui, L. Zong, J. Xie, and M. Tang, "A novel multi-module integrated intrusion detection system for high-dimensional imbalanced data," *Applied Intelligence*, vol. 53, no. 1, pp. 272-288, January 2023.
- [12] I. Jung, J. Ji, and C. Cho, "EmSM: ensemble mixed sampling method for classifying imbalanced intrusion detection data," *Electronics*, vol. 11, no. 9, pp. 1346, 2022.
- [13] N. Gupta, V. Jindal, and P. Bedi, "CSE-IDS: Using cost-sensitive deep learning and ensemble algorithms to handle class imbalance in network-based intrusion detection systems," *Computers & Security*, vol. 112, pp. 102499, 2022.
- [14] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a Dynamic Line Graph Neural Network for Intrusion Detection With Semisupervised Learning," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 699-714, 2023.
- [15] Intrusion Detection Evaluation Dataset (CICIDS2017) (2017). <https://www.unb.ca/cic/datasets/ids-2017.html>
- [16] J. Liu, J. Zheng, J. Wu, and Z. Zheng, "FA-GNN: Filter and Augment Graph Neural Networks for Account Classification in Ethereum," *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 4, pp. 2579-2588, 1 July-Aug 2022.
- [17] M. Xu, B. Zhang, H. Yu, J. Yuan, and C. Wang, "NC-GNN: Consistent Neighbors of Nodes Help More in Graph Neural Networks," *Wireless Communications and Mobile Computing*, 2022.
- [18] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A Comprehensive Survey on Graph Neural Networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4-24, January 2021.
- [19] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2018.
- [20] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, pp. 108-116, 2018.
- [21] G. Karatas, O. Demir, and O. K. Sahingoz, "Increasing the Performance of Machine Learning-Based IDSs on an Imbalanced and Up-to-Date Dataset," *IEEE Access*, vol. 8, pp. 32150-32162, 2020.
- [22] J. Lee, K. Park, "GAN-based imbalanced data intrusion detection system," *Pers Ubiquit Comput* 25, pp. 121-128, 2021.